

Corrections de sécurité à faire

Des tests ont été faits sur l'API Flask (`backend/`) et l'app Angular (`frontend/`). Ce document liste les failles, ce qu'il faut corriger, et surtout : **plus aucune création de compte admin via l'inscription** — l'admin est créé automatiquement à l'init de la base.

Projet : Logiqa **Backend** : `app.py`, `init_db.py` **Frontend** : login, inscription, routes

Règle importante — Compte admin

À appliquer obligatoirement

Supprimer toute possibilité de créer un compte admin / administration depuis l'inscription (frontend + API). Le compte admin **existe déjà** grâce à `backend/init_db.py` au démarrage de `app.py`.

Déjà en place côté base (ne pas recréer à la main)

Au premier lancement, `init_db.py` crée le compte admin s'il n'existe pas :

- Email : `admin@logiqa.local` (ou `ADMIN_EMAIL` dans `.env`)
- Mot de passe : `Admin123!` (ou `ADMIN_PASSWORD` dans `.env`)
- Rôle : `admin`

Ce que tu dois faire

1. **Frontend — inscription** : enlever l'onglet / bouton administration dans `frontend/src/app/inscription/inscription.html` et le type `'administration'` dans `inscription.ts`. Ne plus envoyer `role` dans le POST `/api/register` (ou envoyer seulement `etudiant` / éventuellement `enseignant` si métier, **jamais admin**).
2. **Backend — register** : dans `app.py`, **ignorer** complètement `data.get('role')`. Forcer un rôle non-privilégié, ex. `role = 'etudiant'`. Même si quelqu'un envoie `"role": "admin"` avec curl, ça ne doit **jamais** créer un admin.
3. **Ne pas** ajouter un formulaire “créer admin” ailleurs. L'admin est uniquement seedé dans `init_db.py`.

```
# backend/app.py – register() # AVANT (dangereux) role = data.get('role', 'etudiant') # APRÈS (obligatoire) role = 'etudiant' # jamais depuis le client ; admin uniquement via init_db
```

A. Backend — problèmes à corriger

Critique 1. Élévation de privilèges via `role` à l'inscription

Problème : le client peut envoyer `"role": "admin"` (ou `administration`) dans `POST /api/register`.

```
curl -X POST http://localhost:3000/api/register \ -H "Content-Type: application/json" \ -d '{"nom":"Hack","prenom":"Admin","email":"evil@test.local","password":"Passw0rd!","role":"admin"}'
```

À faire

1. Forcer `role = 'etudiant'` côté serveur (voir section Admin ci-dessus).
2. Vérifier que l'admin seedé dans `init_db.py` fonctionne au login.
3. Ne pas supprimer le seed admin — il remplace la création manuelle / UI.

Moyen 2. XSS stocké dans les champs texte

On peut enregistrer `<script>alert(1)</script>` dans `nom` / `prenom` ; renvoyé dans le JSON de login.

À faire

1. Valider longueur + caractères ; rejeter HTML suspect (400).
2. Validation côté API obligatoire (ne pas compter sur le front).

Moyen 3. Mot de passe trop faible

Password `"1"` accepté.

À faire

- ≥ 8 caractères, au moins 1 lettre et 1 chiffre → sinon HTTP 400.

Moyen 4. Pas de rate limit sur le login

25+ échecs d'affilée acceptés → brute-force possible.

À faire

- Ex. max 5 échecs / 5 min par email ou IP → HTTP 429.

```
from collections import defaultdict from time import time FAILED_LOGINS = defaultdict(list)
MAX_ATTEMPTS = 5 WINDOW_SEC = 300 # purger vieux timestamps → si trop d'échecs return 429 # mauvais
mdp → append ; bon mdp → reset
```

OK SQL paramétré, JWT forgé/expiré rejetés, logout révoque la session — ne pas casser ça.

B. Frontend — problèmes à corriger

Critique 5. Login fake (aucune API)

Fichier : `frontend/src/app/login/login.ts`. N'importe quel login/mot de passe + captcha OK → redirection vers `/dashboard` **sans** appeler `POST /api/login`.

À faire

1. Appeler `POST http://localhost:3000/api/login` avec email + password.
2. En cas de succès : stocker le `token` (ex. `localStorage` ou service Auth).
3. Rediriger vers `/dashboard` **uniquement** si l'API répond OK.
4. Afficher l'erreur API sinon.

Critique 6. Dashboard accessible sans authentification

Fichier : `frontend/src/app/app.routes.ts` — route `/dashboard` sans guard. On peut ouvrir `http://localhost:4200/dashboard` sans être connecté.

À faire

1. Créer un `authGuard` (functional guard Angular).
2. Vérifier qu'un token valide est présent ; sinon redirect `/login`.
3. Protéger toutes les routes privées futures de la même façon.

Haut 7. Choix du rôle « administration » à l'inscription

Fichiers : `inscription.html` (onglets rôle) + `inscription.ts` (envoie `role: this.role()`). Un utilisateur peut s'inscrire comme administration.

À faire (lié à la règle Admin)

1. **Supprimer** le bouton / onglet `administration`.
2. Retirer `'administration'` du type `Role`.
3. Ne plus envoyer un rôle admin au backend. Garder au plus `etudiant` / `enseignant` si besoin métier.
4. Rappeler : l'admin est déjà créé dans la DB via `init_db.py`.

Haut 8. Pas d'interceptor / service auth

Aucun token n'est attaché aux requêtes HTTP. Les futures routes protégées (`/api/me`, `logout`, etc.) échoueront ou resteront ouvertes côté UI.

À faire

1. Créer un `AuthService` (`login`, `logout`, `getToken`, `isLoggedIn`).
2. Ajouter un `HttpInterceptor` qui envoie `Authorization: Bearer <token>`.
3. Sur 401 : clear token + redirect login.

Moyen 9. Captcha 100 % client (inutile pour la sécu)

`login.ts` génère le captcha en JS (`Math.random`) — contournable facilement.

À faire

- Ne pas s'en servir comme seule protection. Le vrai contrôle = API login + rate limit.
- Soit le retirer, soit le remplacer plus tard par un vrai challenge serveur.

Moyen 10. Mot de passe faible côté front + URL API en dur

- Inscription : règle = longueur ≥ 6 seulement.
- URL hardcodée `http://localhost:3000/api/register` — passer par un fichier `environment.ts`.

OK frontend Pas de `innerHTML` / `bypassSecurityTrust*` / secrets en dur — garder l'interpolation Angular `{{ }}`.

Ordre de travail recommandé

1. **Admin** : enlever création admin UI + forcer rôle côté API (seed déjà dans `init_db.py`).

2. **Frontend login réel** branché sur `/api/login` + stockage token.
3. **Auth guard** sur `/dashboard` + interceptor Bearer.
4. Validation mot de passe + champs texte (backend).
5. Rate limit login (backend).
6. Retester checklist ci-dessous.

Comment vérifier

Test	Résultat attendu
Onglet « administration » sur <code>/inscription</code>	Absent
Register avec <code>"role": "admin"</code> (curl)	Compte créé en <code>etudiant</code> (jamais admin)
Login <code>admin@logiqa.local / Admin123!</code>	OK (compte seedé par <code>init_db</code>)
Login avec n'importe quel faux mdp sur le front	Échec (plus de login fake)
Ouvrir <code>/dashboard</code> sans token	Redirect vers <code>/login</code>
Password <code>"1"</code> à l'inscription	HTTP 400
6+ mauvais mots de passe API	HTTP 429

Checklist de livraison

Plus aucun moyen de créer un admin via inscription (UI + API) Admin seedé via `init_db.py` testé (login OK) Login Angular appelle vraiment `/api/login` Guard sur `/dashboard` Interceptor `Authorization: Bearer ...` Mot de passe faible rejeté (API) Champs texte validés (API) Rate limit login actif Happy path inscription étudiant + login OK

Fichiers concernés

- `backend/app.py` — register (rôle forcé), validations, rate limit
- `backend/init_db.py` — seed admin (déjà fait — vérifier / ne pas casser)
- `backend/.env` — `ADMIN_EMAIL`, `ADMIN_PASSWORD`
- `frontend/src/app/login/login.ts` — vrai appel API
- `frontend/src/app/inscription/*` — retirer administration / role admin
- `frontend/src/app/app.routes.ts` — auth guard
- Nouveau : `AuthService` + interceptor HTTP

Questions ? Demande avant de tout refactorer — reste focalisé sur cette liste.